Fully Coupled 1-D Schrodinger–Poisson Solver for a (100) Si MOS Inversion Layer

Including $p(\phi)$, Incomplete Acceptor Ionization $N_A^-(\phi)$, and Quantum Electron Density

Numerical derivation and implementation note

Abstract

This note replaces the earlier strong-inversion approximation $p \approx 0$ by a nonlinear Poisson equation that retains holes, incomplete acceptor ionization, and the quantum electron density. The quantum density is obtained from two (100)-Si valley families with Fermi–Dirac subband occupation. The electrostatic potential is determined by a MOS oxide/gate Robin boundary condition at the interface and a bulk Dirichlet condition. Python and MATLAB implementations are included. The numerical example follows the benchmark conditions used by Janik and Majkusiak: $N_A = 10^{17}\,\mathrm{cm}^{-3}$, $t_{ox} = 3\,\mathrm{nm}$, $T = 300\,\mathrm{K}$, and $V_G = 4\,\mathrm{V}$, while the flat-band voltage remains an explicit model parameter because the paper does not fully specify the absolute gate/oxide/Si band alignment.

1 Physical model

For a planar MOS structure, confinement is along x, normal to the Si/SiO_2 interface. The electron envelope function in valley family v and subband j obeys

$$-\frac{\hbar^2}{2m^*_{z,v}}\frac{d^2\psi_{vj}}{dx^2} + E_c(x)\psi_{vj}(x) = E_{vj}\psi_{vj}(x), \tag{1}$$

with $E_c(x) - E_{c,\mathrm{bulk}} = -q\phi(x)$. For (100) Si there are two valley families:

Family	Degeneracy g_v	confinement mass $m_{z,v}$	in-plane DOS mass $m_{d,v}$
L	2	m_l	m_t
T	4	m_t	$\sqrt{m_l m_t}$

The numerical values used here are $m_l = 0.9163 m_0$ and $m_t = 0.2245 m_0$.

1.1 Fermi–Dirac subband population

The sheet population of subband (v, j) is

$$N_{vj} = \frac{g_v m^*_{d,v} k_B T}{\pi\hbar^2} \ln\left[1 + \exp\left(\frac{E_F - E_{vj}}{k_B T}\right)\right]. \tag{2}$$

The quantum electron concentration is therefore

$$\boxed{n_{\mathrm{QM}}(x) = \sum_{v,j} N_{vj}|\psi_{vj}(x)|^2 + n_0.} \tag{3}$$

The small n_0 term restores the correct equilibrium minority-electron floor in the far bulk.

2 Full nonlinear Poisson equation

The earlier $p \approx 0$ approximation is removed. The charge density is

$$\rho(x) = q\left[p(x) - n_{\mathrm{QM}}(x) - N_A^-(x)\right], \tag{4}$$

so that

$$\boxed{\frac{d^2\phi}{dx^2} = \frac{q}{\varepsilon_{Si}}\left[n_{\mathrm{QM}}(x) + N_A^-(\phi) - p(\phi)\right].} \tag{5}$$

2.1 Hole concentration

With nondegenerate holes,

$$p(\phi) = p_0 \exp\left(-\frac{\phi}{V_T}\right), \qquad V_T = \frac{k_B T}{q}. \tag{6}$$

2.2 Incomplete acceptor ionization

For an acceptor level $E_A = E_V + E_{A0}$ with degeneracy factor g_A,

$$\boxed{N_A^-(\phi) = \frac{N_A}{1 + g_A \exp\left[\frac{E_A(\phi) - E_F}{k_B T}\right]}.} \tag{7}$$

Since the semiconductor bands shift by $-q\phi$ while the equilibrium Fermi level is fixed,

$$E_F - E_V(x) = (E_F - E_V)_{\mathrm{bulk}} + q\phi(x). \tag{8}$$

Hence, in eV units,

$$N_A^-(\phi) = \frac{N_A}{1 + g_A \exp\left\{\frac{E_{A0} - [(E_F - E_V)_{\mathrm{bulk}} + \phi]}{V_T}\right\}}. \tag{9}$$

The implementation uses $E_{A0} = 45\,\mathrm{meV}$ and $g_A = 4$ as a boron-like incomplete-ionization model. These two quantities are an added modeling choice; they are not specified by the attached Janik–Majkusiak paper.

2.3 Bulk charge neutrality

The bulk Fermi level is obtained from

$$p_0 - n_0 - N_{A0}^- = 0, \tag{10}$$

with

$$p_0 = N_V \exp[-(E_F - E_V)/k_B T], \tag{11}$$

$$n_0 = N_C \exp[-(E_C - E_F)/k_B T]. \tag{12}$$

This ensures that the far-bulk Poisson source tends to zero.

3 MOS boundary conditions

At the Si/SiO_2 interface, Gauss' law through the oxide gives the Robin condition

$$\boxed{\left.\frac{d\phi}{dx}\right|_0 = \frac{C_{ox}}{\varepsilon_{Si}}\left[\phi_s - (V_G - V_{FB})\right],} \qquad C_{ox} = \frac{\varepsilon_{ox}}{t_{ox}}. \tag{13}$$

At the far end of the Si domain,

$$\boxed{\phi(L) = 0.} \tag{14}$$

Thus the surface potential is solved rather than imposed.

4 Finite-difference Hamiltonian

On a uniform mesh $x_i = i\Delta x$,

$$\frac{d^2\psi_i}{dx^2} \approx \frac{\psi_{i+1} - 2\psi_i + \psi_{i-1}}{\Delta x^2}. \tag{15}$$

For valley family v, define

$$t_v = \frac{\hbar^2}{2m^*_{z,v}\Delta x^2}. \tag{16}$$

Then the tridiagonal Hamiltonian is

$$H_v = \begin{bmatrix} U_1 + 2t_v & -t_v & 0 & \cdots \\ -t_v & U_2 + 2t_v & -t_v & \cdots \\ 0 & -t_v & U_3 + 2t_v & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}, \tag{17}$$

where $U_i = -\phi_i$ when energies are represented in eV relative to the bulk conduction-band edge.

5 Nonlinear Poisson Newton step

For interior node i,

$$R_i = \frac{\phi_{i-1} - 2\phi_i + \phi_{i+1}}{\Delta x^2} - \frac{q}{\varepsilon_{Si}}\left[n_i + N^-_{A,i}(\phi_i) - p_i(\phi_i)\right]. \tag{18}$$

The diagonal Jacobian contribution is

$$\frac{\partial R_i}{\partial \phi_i} = -\frac{2}{\Delta x^2} - \frac{q}{\varepsilon_{Si}}\left[\frac{dN^-_A}{d\phi} - \frac{dp}{d\phi}\right], \tag{19}$$

where

$$\frac{dp}{d\phi} = -\frac{p}{V_T}, \tag{20}$$

$$\frac{dN^-_A}{d\phi} = \frac{N_A}{V_T} f_A(1 - f_A), \qquad f_A = \frac{N^-_A}{N_A}. \tag{21}$$

For the separate classical comparison, $dn/d\phi = n/V_T$ is added to the Jacobian.

6 Self-consistency algorithm

The quantum density is a nonlocal functional of ϕ because changing the potential changes all eigenvalues and eigenfunctions. The implementation therefore uses a stable nested iteration:

1. Start from the fully nonlinear classical Poisson solution.
2. Solve both valley-family Hamiltonians and assemble $n_{\mathrm{QM}}(x)$.
3. Solve the nonlinear Poisson equation containing n_{QM}, $p(\phi)$, and $N_A^-(\phi)$ in the same residual.
4. Recompute the quantum density from the updated potential.
5. Mix the quantum density geometrically and repeat until the potential update is small.

No charge term is dropped. The charge mixing is only a numerical stabilization of the nonlocal Schrodinger response.

7 Numerical result

For the benchmark-style case $N_A = 10^{17}\,\mathrm{cm}^{-3}$, $t_{ox} = 3\,\mathrm{nm}$, $T = 300\,\mathrm{K}$, $V_G = 4\,\mathrm{V}$, and the exposed parameter $V_{FB} = -0.95\,\mathrm{V}$, the Python calculation converged to a quantum surface potential near $1.468\,\mathrm{V}$. The quantum electron-density maximum occurs about $0.70\,\mathrm{nm}$ from the interface. The exact absolute values depend on the chosen flat-band alignment and ionization model, but the displaced quantum peak is the same qualitative behavior shown in Janik and Majkusiak's Fig. 3.

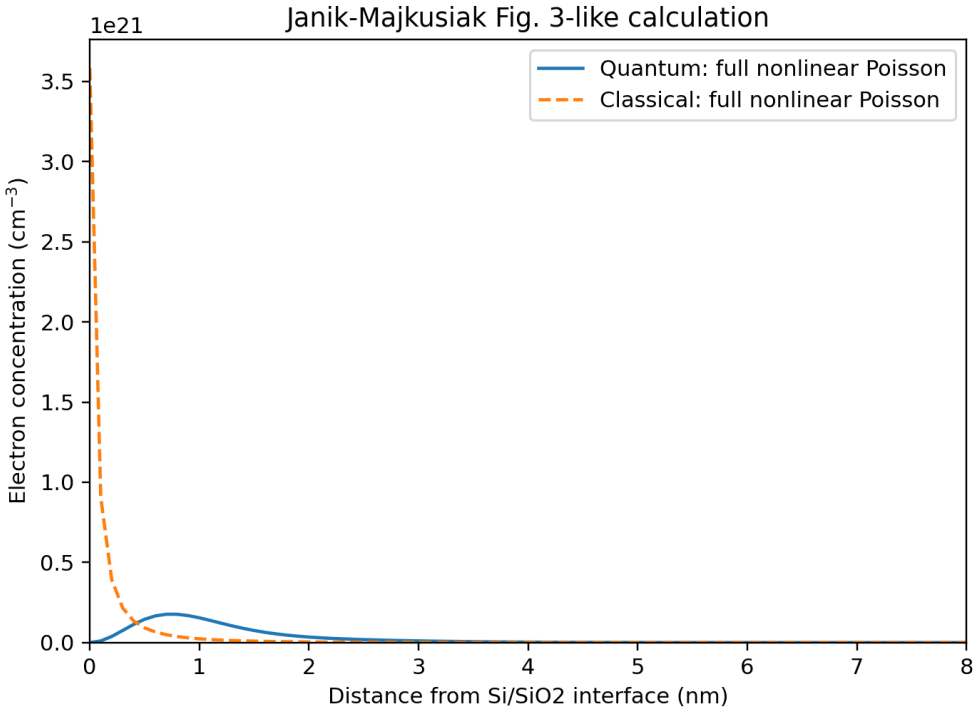


Figure 1: Fully nonlinear classical and quantum electron distributions.

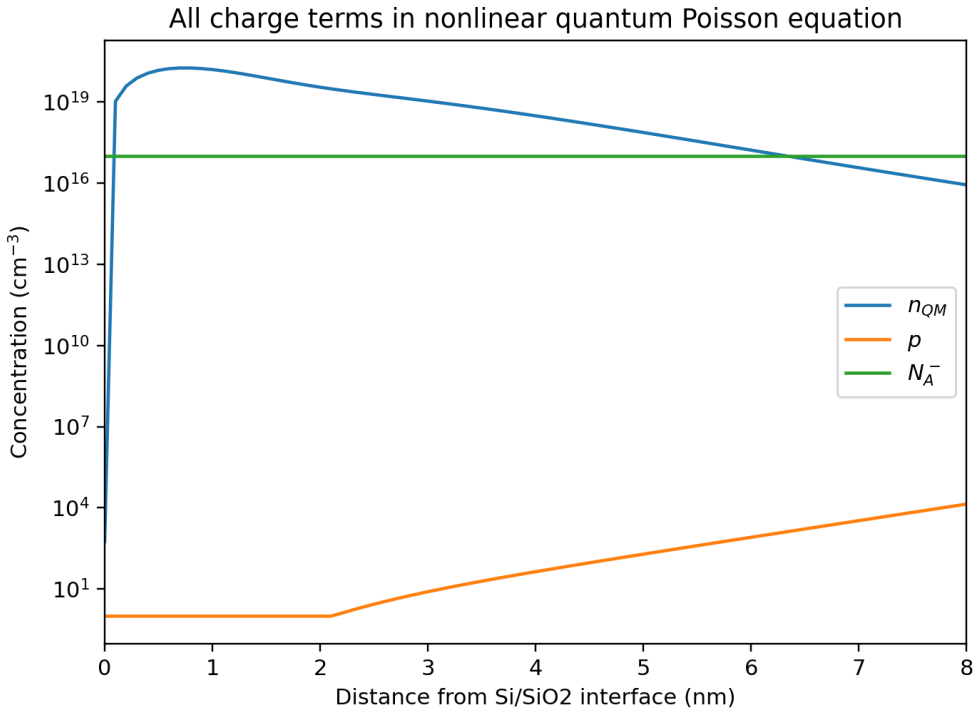


Figure 2: Quantum Poisson charge terms n_{QM}, p, and N_A^-. Holes are strongly suppressed in inversion but are retained rather than set to zero.

8 Relation to the attached paper

Janik and Majkusiak formulate the electrostatics of the two-dimensional electron gas through a self-consistent one-dimensional Schrodinger–Poisson system, solve separate valley families for (100) Si, and compare quantum and classical electron profiles. Their Fig. 3 shows the classical maximum at the interface and the quantum maximum shifted into Si. They also state that rigorous numerical Schrodinger–Poisson analysis is required for accurate surface-potential calculations in ultrathin-oxide MOS structures. The present implementation extends the earlier code by retaining holes and incomplete acceptor ionization throughout the nonlinear Poisson solve.

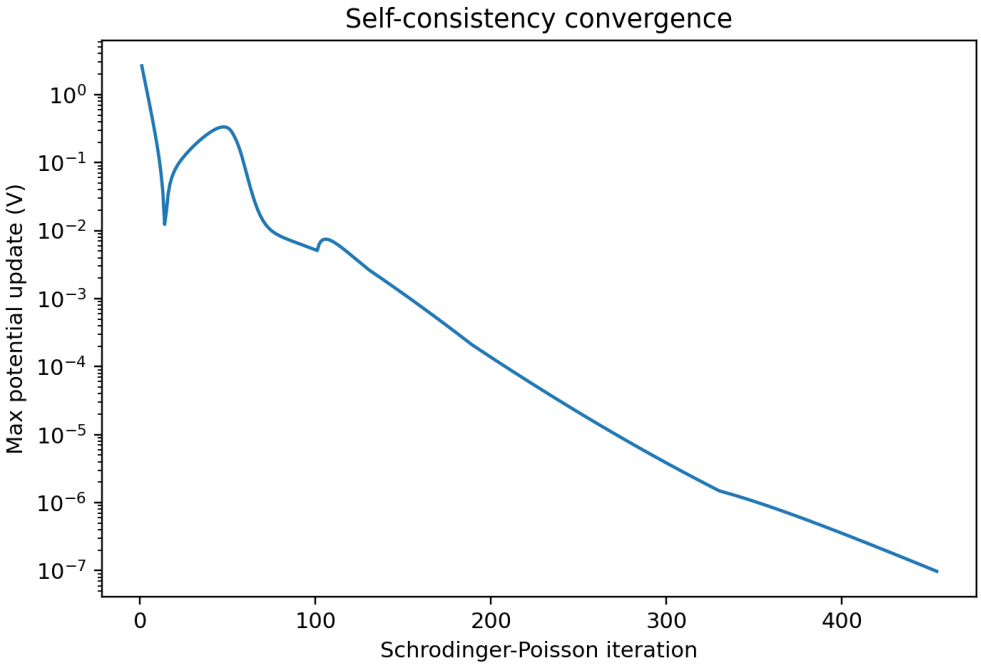


Figure 3: Outer Schrodinger–Poisson convergence history.

A Python implementation

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import eigh_tridiagonal
from scipy.integrate import solve_bvp
from scipy.optimize import brentq
from scipy.sparse import lil_matrix
from scipy.sparse.linalg import spsolve

# Fully coupled 1-D MOS Schrodinger-Poisson solver
# (100) Si, two valley families, Fermi-Dirac subband occupation,
# holes and incomplete acceptor ionization retained in Poisson equation.

q = 1.602176634e-19
hbar = 1.054571817e-34
kB = 1.380649e-23
m0 = 9.1093837015e-31
eps0 = 8.8541878128e-12

T = 300.0
VT = kB*T/q
eps_si = 11.7*eps0
eps_ox = 3.9*eps0
Eg = 1.122 # eV
NA = 1.0e17*1e6 # m^-3
tox = 3.0e-9 # m
Vg = 4.0 # V
Vfb = -0.95 # V, exposed alignment parameter

# (100) silicon effective masses used in Janik-Majkusiak paper
ml = 0.9163*m0
mt = 0.2245*m0
# name, valley degeneracy, confinement mass, in-plane DOS mass
valleys = [('L', 2, ml, mt), ('T', 4, mt, np.sqrt(ml*mt))]

# Bulk DOS values. Nc from ellipsoidal valley masses; Nv conventional 300 K value.
md3 = (ml*mt*mt)**(1/3)
Nc = 2*6*(md3*kB*T/(2*np.pi*hbar**2))**1.5
Nv = 1.04e19*1e6
ni = np.sqrt(Nc*Nv)*np.exp(-Eg/(2*VT))

# Boron-like acceptor model: acceptor level Ea = Ev + EA_ION.
EA_ION = 0.045 # eV
GA = 4.0 # acceptor degeneracy factor

# Determine equilibrium bulk Fermi level with incomplete ionization.
# eta_v = (Ef-Ev)/kT in volts/eV divided by VT.
def bulk_neutrality(EfEv):
    p = Nv*np.exp(-EfEv/VT)
    n = Nc*np.exp(-(Eg-EfEv)/VT)
    NAminus = NA/(1.0 + GA*np.exp((EA_ION-EfEv)/VT))
    return p - n - NAminus

EfEv = brentq(bulk_neutrality, 0.001, 0.5)
EcEf = Eg - EfEv
p0 = Nv*np.exp(-EfEv/VT)
n0 = Nc*np.exp(-EcEf/VT)
NA0minus = NA/(1.0 + GA*np.exp((EA_ION-EfEv)/VT))

# Numerical mesh
L = 30e-9
N = 301
x = np.linspace(0.0, L, N)
dx = x[1]-x[0]
Ni = N-2
alpha = eps_ox/(eps_si*tox)
```

```
def log1pexp(a):
    a = np.asarray(a, dtype=float)
    out = np.empty_like(a)
    hi = a > 40.0
    lo = a < -40.0
    mid = ~(hi | lo)
    out[hi] = a[hi]
    out[lo] = np.exp(a[lo])
    out[mid] = np.log1p(np.exp(a[mid]))
    return out


def holes(phi):
    """Nondegenerate hole density p(phi), exact bulk value at phi=0."""
    return p0*np.exp(np.clip(-phi/VT, -100.0, 100.0))


def ionized_acceptors(phi):
    """Incomplete ionization N_A^-(phi).

    Electrostatic potential raises Ef-Ev locally by +phi (eV), since both
    semiconductor band edges shift by -q phi while Ef is fixed at equilibrium.
    """
    local_EfEv = EfEv + phi
    arg = np.clip((EA_ION-local_EfEv)/VT, -100.0, 100.0)
    return NA/(1.0 + GA*np.exp(arg))


def classical_electrons(phi):
    """Classical MB electron density for the separate reference solution."""
    return n0*np.exp(np.clip(phi/VT, -100.0, 80.0))


def schrodinger_density(phi, nstates=10):
    """Quantum electron density from two Si valley families.

    Energies are measured relative to the bulk conduction-band edge Ec_bulk.
    U(x) = Ec(x)-Ec_bulk = -phi(x) in eV.
    """
    U = -phi
    ntot = np.zeros_like(x)
    info = []
    for name, g, mz, md in valleys:
        t = (hbar*hbar/(2.0*mz*dx*dx))/q # eV
        diag = U[1:-1] + 2.0*t
        off = -t*np.ones(Ni-1)
        ns = min(nstates, Ni)
        vals, vecs = eigh_tridiagonal(diag, off, select='i', select_range=(0, ns-1))
        sheets = []
        psis = []
        for E, v in zip(vals, vecs.T):
            psi = np.zeros(N)
            psi[1:-1] = v
            psi /= np.sqrt(np.trapezoid(psi*psi, x))
            eta = ((-EcEf)-E)/VT # (Ef - Esub)/kT
            Ns = g*md*kB*T/(np.pi*hbar*hbar)*log1pexp([eta])[0]
            ntot += Ns*psi*psi
            sheets.append(Ns)
            psis.append(psi)
        info.append({'name': name, 'g': g, 'mz': mz, 'md': md,
                     'E': np.array(vals), 'Ns': np.array(sheets),
                     'psi': np.array(psis)})

    # Add the equilibrium minority density as a smooth far-bulk floor.
    ntot += n0
    return ntot, info


def gate_bc(ya, yb):
    # epsilon_si phi'(0) = Cox[phi_s-(Vg-Vfb)] ; phi(L)=0
```

```
    return np.array([ya[1] - alpha*(ya[0]-(Vg-Vfb)), yb[0]])


def solve_poisson_nonlinear(n_fixed, guess, tol=1e-7, maxit=100):
    """Damped-Newton finite-difference Poisson solve.

    Residual contains all local charge terms simultaneously:
      phi'' - q/eps_si [n_fixed + N_A^-(phi) - p(phi)] = 0.
    The quantum density is nonlocal in phi and is refreshed by the outer
    Schrodinger-Poisson iteration.
    """
    M = N-1 # phi[0:M] unknown, phi[N-1]=0 bulk Dirichlet
    ph = guess[:M].copy()
    Vgb = Vg - Vfb
    for _ in range(maxit):
        full = np.zeros(N); full[:M] = ph
        pp = holes(full)[:M]
        naa = ionized_acceptors(full)[:M]
        # derivatives wrt phi
        dpp = -pp/VT
        f = naa/NA
        dna = NA*f*(1.0-f)/VT

        R = np.zeros(M)
        J = lil_matrix((M,M))
        # second-order one-sided gate derivative
        R[0] = (-3*ph[0] + 4*ph[1] - ph[2])/(2*dx) - alpha*(ph[0]-Vgb)
        J[0,0] = -3/(2*dx) - alpha
        J[0,1] = 4/(2*dx)
        J[0,2] = -1/(2*dx)
        for i in range(1,M):
            pim1 = ph[i-1]
            pi = ph[i]
            pip1 = ph[i+1] if i+1<M else 0.0
            R[i] = (pim1 - 2*pi + pip1)/dx**2 - q/eps_si*(n_fixed[i] + naa[i] - pp[i])
            J[i,i-1] = 1/dx**2
            J[i,i] = -2/dx**2 - q/eps_si*(dna[i] - dpp[i])
            if i+1<M: J[i,i+1] = 1/dx**2
        if np.max(np.abs(R[1:])) < 1e10 and abs(R[0]) < 1e3:
            break
        delta = spsolve(J.tocsr(), -R)
        # line search on scaled update; restrict excessively large Newton moves
        maxdu = np.max(np.abs(delta))
        if maxdu > 0.15: delta *= 0.15/maxdu
        lam = 1.0
        norm0 = np.linalg.norm(R)
        accepted = False
        for _ls in range(12):
            trial = ph + lam*delta
            tfull=np.zeros(N); tfull[:M]=trial
            tpp=holes(tfull)[:M]; tna=ionized_acceptors(tfull)[:M]
            Rt=np.zeros(M)
            Rt[0]=(-3*trial[0]+4*trial[1]-trial[2])/(2*dx)-alpha*(trial[0]-Vgb)
            for i in range(1,M):
                pip1=trial[i+1] if i+1<M else 0.0
                Rt[i]=(trial[i-1]-2*trial[i]+pip1)/dx**2-q/eps_si*(n_fixed[i]+tna[i]-tpp[i])
            if np.linalg.norm(Rt) < norm0:
                ph=trial; accepted=True; break
            lam*=0.5
        if not accepted:
            ph += 0.05*delta
        if np.max(np.abs(lam*delta)) < tol:
            break
    out=np.zeros(N); out[:M]=ph
    return out


def solve_classical(guess=None, tol=1e-7, maxit=150):
    if guess is None:
        guess = 0.9*np.exp(-x/5e-9)
```

```
    M=N-1; ph=guess[:M].copy(); Vgb=Vg-Vfb
    for _ in range(maxit):
        full=np.zeros(N); full[:M]=ph
        nn=classical_electrons(full)[:M]
        pp=holes(full)[:M]
        naa=ionized_acceptors(full)[:M]
        dnn=nn/VT; dpp=-pp/VT
        f=naa/NA; dna=NA*f*(1-f)/VT
        R=np.zeros(M); J=lil_matrix((M,M))
        R[0]=(-3*ph[0]+4*ph[1]-ph[2])/(2*dx)-alpha*(ph[0]-Vgb)
        J[0,0]=-3/(2*dx)-alpha; J[0,1]=4/(2*dx); J[0,2]=-1/(2*dx)
        for i in range(1,M):
            pip1=ph[i+1] if i+1<M else 0.0
            R[i]=(ph[i-1]-2*ph[i]+pip1)/dx**2-q/eps_si*(nn[i]+naa[i]-pp[i])
            J[i,i-1]=1/dx**2
            J[i,i]=-2/dx**2-q/eps_si*(dnn[i]+dna[i]-dpp[i])
            if i+1<M: J[i,i+1]=1/dx**2
        delta=spsolve(J.tocsr(),-R)
        maxdu=np.max(np.abs(delta))
        if maxdu>0.10: delta*=0.10/maxdu
        ph += 0.5*delta
        if np.max(np.abs(0.5*delta))<tol: break
    out=np.zeros(N); out[:M]=ph
    return out


# Classical reference
phi_c = solve_classical()
nc = classical_electrons(phi_c)
pc = holes(phi_c)
NAc = ionized_acceptors(phi_c)

# Quantum self-consistency with charge mixing.
# The nonlinear Poisson solve contains p(phi), N_A^-(phi), and the current
# quantum density in the same residual. The nonlocal Schrodinger response is
# stabilized by mixing n_QM rather than deleting any charge term.
phi_q = phi_c.copy()
n_work, info = schrodinger_density(phi_q)
history = []
for it in range(600):
    phi_raw = solve_poisson_nonlinear(n_work, phi_q, tol=1e-7)
    nq_new, info = schrodinger_density(phi_raw)
    err = np.max(np.abs(phi_raw-phi_q))
    history.append(err)
    beta = 0.01 if it < 100 else 0.02
    # logarithmic/geometric mixing is robust over many decades of density
    floor = max(n0, 1.0)
    logn = (1-beta)*np.log(np.maximum(n_work, floor)) + beta*np.log(np.maximum(nq_new, floor))
    n_work = np.exp(logn)
    phi_q = 0.25*phi_raw + 0.75*phi_q
    if err < 2e-5 and np.max(np.abs(np.log(np.maximum(nq_new,floor)/np.maximum(n_work,floor)))) < 2e-3:
        break
else:
    print('Warning: outer Schrodinger-Poisson loop hit iteration limit')

nq, info = schrodinger_density(phi_q)
pq = holes(phi_q)
NAq = ionized_acceptors(phi_q)

# Diagnostics
xmax = x[np.argmax(nq)]
xavq = np.trapezoid(x*nq, x)/np.trapezoid(nq, x)
xavc = np.trapezoid(x*nc, x)/np.trapezoid(nc, x)
net_q = nq + NAq - pq

print(f'Bulk Ef-Ev = {EfEv:.6f} eV, Ec-Ef = {EcEf:.6f} eV')
print(f'Bulk p = {p0/1e6:.4e} cm^-3, n = {n0/1e6:.4e} cm^-3, NA- = {NAOminus/1e6:.4e} cm^-3')
print(f'Bulk charge-neutrality residual = {(p0-n0-NAOminus)/1e6:.4e} cm^-3')
print(f'Quantum iterations = {len(history)}, final max update = {history[-1]:.3e} V')
print(f'phi_s quantum = {phi_q[0]:.6f} V, classical = {phi_c[0]:.6f} V')
```

```
print(f'quantum xmax = {xmax*1e9:.3f} nm, xav_Q = {xavq*1e9:.3f} nm, xav_C = {xavc*1e9:.3f} nm')
for a in info:
    print(a['name'], 'first 6 E [eV rel Ec_bulk]=', np.round(a['E'][:6], 5))

# Save data
out = np.c_[x*1e9, phi_q, phi_c, nq/1e6, nc/1e6, pq/1e6, pc/1e6, NAq/1e6, NAc/1e6]
np.savetxt('/mnt/data/janik_full_nonlinear_output.csv', out, delimiter=',',
           header='x_nm,phi_quantum_V,phi_classical_V,n_quantum_cm-3,n_classical_cm-3,p_quantum_cm-3,p_classical_cm
               -3,NAminus_quantum_cm-3,NAminus_classical_cm-3', comments='')

# Fig. 3-like profile
m = x <= 8e-9
plt.figure(figsize=(6.5, 4.8))
plt.plot(x[m]*1e9, nq[m]/1e6, label='Quantum: full nonlinear Poisson')
plt.plot(x[m]*1e9, nc[m]/1e6, '--', label='Classical: full nonlinear Poisson')
plt.xlabel('Distance from Si/SiO2 interface (nm)')
plt.ylabel('Electron concentration (cm$^{-3}$)')
plt.title('Janik-Majkusiak Fig. 3-like calculation')
plt.xlim(0, 8)
plt.ylim(bottom=0)
plt.legend()
plt.tight_layout()
plt.savefig('/mnt/data/janik_full_nonlinear_fig3.png', dpi=220)
plt.close()

# Carrier and ionized acceptor profiles
plt.figure(figsize=(6.5, 4.8))
plt.semilogy(x[m]*1e9, np.maximum(nq[m]/1e6, 1), label='$n_{QM}$')
plt.semilogy(x[m]*1e9, np.maximum(pq[m]/1e6, 1), label='$p$')
plt.semilogy(x[m]*1e9, np.maximum(NAq[m]/1e6, 1), label='$N_A^-$')
plt.xlabel('Distance from Si/SiO2 interface (nm)')
plt.ylabel('Concentration (cm$^{-3}$)')
plt.title('All charge terms in nonlinear quantum Poisson equation')
plt.xlim(0, 8)
plt.legend()
plt.tight_layout()
plt.savefig('/mnt/data/janik_full_charge_terms.png', dpi=220)
plt.close()

# Convergence history
plt.figure(figsize=(6.5, 4.5))
plt.semilogy(np.arange(1, len(history)+1), history)
plt.xlabel('Schrodinger-Poisson iteration')
plt.ylabel('Max potential update (V)')
plt.title('Self-consistency convergence')
plt.tight_layout()
plt.savefig('/mnt/data/janik_full_nonlinear_convergence.png', dpi=220)
plt.close()
```

B MATLAB implementation

```
% Fully nonlinear 1-D Schrodinger-Poisson MOS solver
% Retains p(phi), incomplete acceptor ionization N_A^-(phi), and n_QM(x).
clear; clc;
q=1.602176634e-19; hbar=1.054571817e-34; kB=1.380649e-23;
m0=9.1093837015e-31; eps0=8.8541878128e-12;
T=300; VT=kB*T/q; eps_si=11.7*eps0; eps_ox=3.9*eps0;
Eg=1.122; NA=1e17*1e6; tox=3e-9; Vg=4.0; Vfb=-0.95;
ml=0.9163*m0; mt=0.2245*m0; Nv=1.04e19*1e6;
md3=(ml*mt*mt)^(1/3); Nc=2*6*(md3*kB*T/(2*pi*hbar^2))^(3/2);
EA=0.045; gA=4;
fun=@(EfEv) Nv*exp(-EfEv/VT)-Nc*exp(-(Eg-EfEv)/VT) ...
    - NA/(1+gA*exp((EA-EfEv)/VT));
EfEv=fzero(fun,[0.001 0.5]); EcEf=Eg-EfEv;
p0=Nv*exp(-EfEv/VT); n0=Nc*exp(-EcEf/VT);
L=30e-9; N=301; x=linspace(0,L,N)'; dx=x(2)-x(1);
alpha=eps_ox/(eps_si*tox);

holes=@(phi) p0.*exp(max(min(-phi/VT,100),-100));
NAminus=@(phi) NA./(1+gA.*exp(max(min((EA-(EfEv+phi))/VT,100),-100)));
nclass=@(phi) n0.*exp(max(min(phi/VT,80),-100));

phi_c=0.9*exp(-x/5e-9);
for it=1:150
    [phi_new,du]=poisson_newton(phi_c,nclass(phi_c),true,x,dx,alpha,Vg,Vfb,q,eps_si,holes,NAminus,nclass,VT,NA);
    phi_c=phi_new; if du<1e-7, break; end
end
nc=nclass(phi_c);

phi_q=phi_c; [nwork,levels]=qdensity(phi_q,x,dx,hbar,q,kB,T,ml,mt,EcEf,n0);
hist=[];
for it=1:600
    [phi_raw,~]=poisson_newton(phi_q,nwork,false,x,dx,alpha,Vg,Vfb,q,eps_si,holes,NAminus,nclass,VT,NA);
    [nnew,levels]=qdensity(phi_raw,x,dx,hbar,q,kB,T,ml,mt,EcEf,n0);
    err=max(abs(phi_raw-phi_q)); hist(end+1)=err;
    if it<100, beta=0.01; else, beta=0.02; end
    floorv=max(n0,1); nwork=exp((1-beta)*log(max(nwork,floorv))+beta*log(max(nnew,floorv)));
    phi_q=0.25*phi_raw+0.75*phi_q;
    if err<2e-5, break; end
end
[nq,levels]=qdensity(phi_q,x,dx,hbar,q,kB,T,ml,mt,EcEf,n0);
pq=holes(phi_q); NAq=NAminus(phi_q);
[~,im]=max(nq); fprintf('phi_s(Q)=%.6f V, xmax=%.3f nm\n',phi_q(1),x(im)*1e9);

mask=x<=8e-9; figure; plot(x(mask)*1e9,nq(mask)/1e6,'LineWidth',1.5); hold on;
plot(x(mask)*1e9,nc(mask)/1e6,'--','LineWidth',1.5); xlabel('Distance (nm)');
ylabel('Electron concentration (cm^{-3})'); legend('Quantum','Classical'); grid on;
writematrix([x*1e9 phi_q phi_c nq/1e6 nc/1e6 pq/1e6 NAq/1e6], 'janik_full_nonlinear_matlab.csv');

function [phi,du]=poisson_newton(guess,nfix,isclass,x,dx,alpha,Vg,Vfb,q,eps_si,holes,NAminus,nclass,VT,NA)
N=length(x); M=N-1; ph=guess(1:M); Vgb=Vg-Vfb;
for it=1:120
  full=[ph;0]; pp=holes(full); naa=NAminus(full);
  if isclass, nn=nclass(full); else, nn=nfix; end
  R=zeros(M,1); J=spalloc(M,M,3*M);
  R(1)=(-3*ph(1)+4*ph(2)-ph(3))/(2*dx)-alpha*(ph(1)-Vgb);
  J(1,1)=-3/(2*dx)-alpha; J(1,2)=4/(2*dx); J(1,3)=-1/(2*dx);
  dpp=-pp/VT; frac=naa/NA; dna=NA.*frac.*(1-frac)/VT;
  if isclass, dnn=nn/VT; else, dnn=zeros(size(nn)); end
  for i=2:M
    pip1=0; if i<M, pip1=ph(i+1); end
    R(i)=(ph(i-1)-2*ph(i)+pip1)/dx^2-q/eps_si*(nn(i)+naa(i)-pp(i));
    J(i,i-1)=1/dx^2; J(i,i)=-2/dx^2-q/eps_si*(dnn(i)+dna(i)-dpp(i));
    if i<M, J(i,i+1)=1/dx^2; end
  end
  d=J\(-R); if max(abs(d))>0.1, d=d*(0.1/max(abs(d))); end
  ph=ph+0.5*d; du=max(abs(0.5*d)); if du<1e-7, break; end
end
```

```
phi=[ph;0];
end

function [ntot,levels]=qdensity(phi,x,dx,hbar,q,kB,T,ml,mt,EcEf,n0)
N=length(x); Ni=N-2; U=-phi; ntot=zeros(N,1); nstates=10;
V={2,ml,mt;4,mt,sqrt(ml*mt)}; levels=cell(2,1); VT=kB*T/q;
for iv=1:2
 g=V{iv,1}; mz=V{iv,2}; md=V{iv,3}; t=(hbar^2/(2*mz*dx^2))/q;
 main=U(2:end-1)+2*t; off=-t*ones(Ni-1,1); H=diag(main)+diag(off,1)+diag(off,-1);
 [vec,D]=eigs(H,nstates,'smallestreal'); E=diag(D); [E,ord]=sort(E); vec=vec(:,ord);
 for j=1:nstates
   psi=zeros(N,1); psi(2:end-1)=vec(:,j); psi=psi/sqrt(trapz(x,psi.^2));
   eta=((-EcEf)-E(j))/VT; if eta>40,l=eta;elseif eta<-40,l=exp(eta);else,l=log1p(exp(eta));end
   Ns=g*md*kB*T/(pi*hbar^2)*l; ntot=ntot+Ns*psi.^2;
 end
 levels{iv}=E;
end
ntot=ntot+n0;
end
```

C References

References

[1] T. Janik and B. Majkusiak, "Analysis of the MOS Transistor Based on the Self-Consistent Solution to the Schrodinger and Poisson Equations and on the Local Mobility Model," *IEEE Transactions on Electron Devices*, vol. 45, no. 6, pp. 1263–1271, June 1998.